String Indexing (Exercises)

1. `'python'[0]` 'p'
2. `'python'[1]` _____
3. `'python'[3]` _____
4. `'python'[-1]` _____
5. `'python'[-3]` _____

String Indexing

Strings of text may be surrounded by single quotation marks (`'`).

Putting a number in square brackets (`[]`) after a string gives the single character at that *index* (position). Indices are zero based, so the first character is at index 0.

Negative indices start from the end of the string, so index -1 is the last character, index -2 is the character before that, and so on.

String Slicing (Exercises)

1. `'abandon'[0:2]` 'ab'
2. `'abandon'[1:5]` _____
3. `'infinite'[2:100]` _____
4. `'incandescent'[3:8:2]` _____
5. `'airspace'[:3]` _____
6. `'airspace'[3:]` _____
7. `'remote'[4:0:-1]` _____
8. `'duplicate'[::]` _____
9. `'straw'[::-1]` _____

String Slicing

Up to three numbers separated by colons (`:`) within square brackets after a string give a *slice* of the string (a shorter string):

- The first number gives the *start* index of the slice.
- The second number gives the *stop* index. This is exclusive, so the slice includes character up to but not including the stop index.
- The third number gives the *step*. The index is advanced by this much for each character of the slice, so (for example) a step of 2 will include every other character. If the step is negative, the slice works backward from the start to (but not including) the stop.

Any or all of the three numbers specifying a slice may be omitted.

- If no step is provided, the slice starts at the beginning of the string.
    - ... unless the step is negative, in which case the slice starts at the end of the string.
- If no stop is provided, the slice stops at the end of the string.
    - ... unless the step is negative, in which case the slice stops at the beginning of the string.
- If no step is provided, a step of 1 is used. The second colon can be omitted in this case.

If the stop is beyond the end of the string, the slice ends where the string ends.

Arithmetic Operators (Exercises)

1. `2 + 2` 4
2. `2 + 3` _____
3. `4 - 1` _____
4. `3 - 8` _____
5. `2 * 4` _____
6. `8 / 2` _____
7. `7 / 2` _____
8. `7 // 2` _____
9. `34 // 10` _____
10. `34 % 10` _____
11. `2 ** 3` _____
12. `10 ** 20` _____
13. `2 + 3 * 5` _____
14. `2 + (3 * 5)` _____
15. `(2 + 3) * 5` _____
16. `1 + 2 * 3 - 4 ** 2 / 4` _____

Arithmetic Operators

Numbers can be combined with familiar arithmetic *operators*. `+` and `-` perform addition and subtraction. The multiplication operator is `*`.

There are three division operators:

- `/` performs standard division. The answer always includes a decimal point, even if it happens to be an integer.
- `//` performs integer division, throwing away any remainder.
- `%`, the *modulo* or *remainder* operator, keeps *only* the remainder.

The `**` operator performs exponentiation, so `5 ** 2` computes 5^2.

Some operators have higher precedence than others, so they are performed first:

- `**` has the highest precedence
- `*`, `/`, `//`, and `%` have the next highest
- `+` and `-` have the lowest

Parentheses can be used to ensure that particular operations are performed earlier.

Variables (Exercises)

1.
```
x = 3
```
```
x
```
3

2.
```
x + 2
```

3.
```
y = 4
```
```
y
```

4.
```
x + y
```

5.
```
color = 'blue'
```
```
color
```

6.
```
score = 5
score = 10
```
```
score
```

7.
```
a = 1
b = a
a = 2
```
```
b
```

8.
```
n = 10
n = n + 1
```
```
n
```

Variables

A *variable* is a name for something. It can be *assigned* a value with the = operator. For example, `x = 3` makes `x` a name for the number 3.

Once it has been assigned a value, the variable can be used in place of the value.

A variable can be redefined by assigning it a new value.

Note that = is not a declaration that two things are the same. It is an assignment that makes the variable name on the left a name for the value on the right.

Assignment makes the variable stand for the *value* on the right side, not the code that produced that value. Specifically:

- If a variable is defined in terms of another variable, later changes to the other variable don't affect the first one.
- A variable can be defined in terms of its current value. For example, `x = x + 1` gets the current value of `x`, adds 1, and makes `x` a name for that new value.

String Operators (Exercises)

1. `'hot' + 'dog'` 'hotdog'
2. `'shoe' + 'string'` _____
3. `'ma' * 2` _____
4. `'co' * 2 + 'a'` _____
5. `'i' in 'team'` _____
6. `'on' in ('no' * 2)` _____
7.
```
a = 'brown'
b = 'toad'
a[1:3] + b[2:]
```
_____

String Operators

Two strings can be concatenated with `+`.

Multiplying a string by a positive integer (using `*`) concatenates multiple copies of it.

If `a` and `b` are strings, `a in b` is true if the string `a` appears somewhere within `b`.

Quotation Marks (Exercises)

1. `"hello"` `'hello'`
2. `'goodbye'` _____
3. `"don't"` _____
4. `'Eddie "The Eagle" Edwards'` _____
5. `'You "can\'t" avoid it.'` _____
6. `"... or \"can\" you?"` _____
7. `'Maybe \'sometimes\' you can.'` _____

Quotation Marks

Strings may be delimited by either single (`'`) or double (`"`) quotation marks.

Single quotation marks can be used in a string delimited by double quotation marks, and vice versa.

If it is necessary to use the kind of quotation mark that delimits a string inside that string, precede it with a backslash (`\`).

When it displays strings, Python prefers single quotation marks, but will use double quotation marks to avoid a backslash.

Calling Functions (Exercises)

1. `abs(-3)` 3
2. `abs(12)` _____
3. `len('python')` _____
4. `max(2, 3)` _____
5. `min(2, 3)` _____
6. `round(3.14159)` _____
7.
```
a = 'snake'
b = 'language'
max(len(a), len(b))
```
_____
8.
```
s = 'python'
s[0:len(s)]
```
_____
9. `max(1, min(2, 3))` _____

Calling Functions

To *call* a *function*, use the name of the function followed by round parentheses (`()`). If the function takes any *arguments*, put them between the parentheses, separated by commas. The value returned by the function call depends on the function and the arguments.

Here are some of Python's built-in functions:

- `abs(x)` returns the absolute value of the number `x`.
- `len(s)` returns the length of the string `s`.
- `max(x, y)` returns the larger of `x` and `y`.
- `min(x, y)` returns the smaller of `x` and `y`.
- `round(x)` returns the number `x` rounded to the nearest integer. If `x` is exactly halfway between two integers, `round` returns the even one.

Many of these functions can take additional, optional arguments. Many of them work similarly on other kinds of values, to be explained later.

Lists (Exercises)

1. `['eggs', 'bread', 'tea']` `['eggs', 'bread', 'tea']`
2. `nums = [4, 8, 15, 16, 23, 42]`

   `nums[3]` _____
3. `nums[2:4]` _____
4. `nums[1] = 99`

   `nums` _____
5. `len(nums)` _____
6. `nest = [[6, 7], [1, 1, 3, 8]]`

   `nest[1][2]` _____
7. `left = [1, 2, 3]`

   `right = [4, 5, 6]`

   `left + right` _____
8. `left * 2` _____
9. `3 in left` _____
10. `[2, 3] in left` _____
11. `alias = left`

    `left[0] = 10`

    `alias` _____

Lists

A *list* is a sequence of elements, delimited by square brackets `[` and `]`. The elements might be strings, numbers, or even other lists.

Lists behave much like strings. They can be accessed or sliced in exactly the same way.

Unlike strings, lists can be modified. For example, if `ingredients` is a list, `ingredients[0] = 'olive oil'` replaces the first element of the list. This can lead to surprises when a list has more than one name.

The `in` operator also behaves slightly differently. For a list `ls`, `a in ls` is true if `a` is an element of `ls`. For a string `s`, `a in s` is true if `a` is a substring of `s`.

String Indexing (Solutions)

1. `'python'[0]` 'p'
2. `'python'[1]` 'y'
3. `'python'[3]` 'h'
4. `'python'[-1]` 'n'
5. `'python'[-3]` 'h'

String Slicing (Solutions)

1. `'abandon'[0:2]` `'ab'`
2. `'abandon'[1:5]` `'band'`
3. `'infinite'[2:100]` `'finite'`
4. `'incandescent'[3:8:2]` `'ads'`
5. `'airspace'[:3]` `'air'`
6. `'airspace'[3:]` `'space'`
7. `'remote'[4:0:-1]` `'tome'`
8. `'duplicate'[::]` `'duplicate'`
9. `'straw'[::-1]` `'warts'`

Arithmetic Operators (Solutions)

1. `2 + 2` 4
2. `2 + 3` 5
3. `4 - 1` 3
4. `3 - 8` -5
5. `2 * 4` 8
6. `8 / 2` 4.0
7. `7 / 2` 3.5
8. `7 // 2` 3
9. `34 // 10` 3
10. `34 % 10` 4
11. `2 ** 3` 8
12. `10 ** 20` 100000000000000000000
13. `2 + 3 * 5` 17
14. `2 + (3 * 5)` 17
15. `(2 + 3) * 5` 25
16. `1 + 2 * 3 - 4 ** 2 / 4` 3.0

Variables (Solutions)

1.
```
x = 3
x
```
3

2.
```
x + 2
```
5

3.
```
y = 4
y
```
4

4.
```
x + y
```
7

5.
```
color = 'blue'
color
```
'blue'

6.
```
score = 5
score = 10
score
```
10

7.
```
a = 1
b = a
a = 2
b
```
1

8.
```
n = 10
n = n + 1
n
```
11

String Operators (Solutions)

1. `'hot' + 'dog'` `'hotdog'`
2. `'shoe' + 'string'` `'shoestring'`
3. `'ma' * 2` `'mama'`
4. `'co' * 2 + 'a'` `'cocoa'`
5. `'i' in 'team'` `False`
6. `'on' in ('no' * 2)` `True`
7.
```
a = 'brown'
b = 'toad'
a[1:3] + b[2:]
```
`'road'`

Quotation Marks (Solutions)

1. `"hello"` `'hello'`
2. `'goodbye'` `'goodbye'`
3. `"don't"` `"don't"`
4. `'Eddie "The Eagle" Edwards'` `'Eddie "The Eagle" Edwards'`
5. `'You "can\'t" avoid it.'` `'You "can\'t" avoid it.'`
6. `"... or \"can\" you?"` `'... or "can" you?'`
7. `'Maybe \'sometimes\' you can.'` `"Maybe 'sometimes' you can."`

Calling Functions (Solutions)

1. `abs(-3)` 3
2. `abs(12)` 12
3. `len('python')` 6
4. `max(2, 3)` 3
5. `min(2, 3)` 2
6. `round(3.14159)` 3
7.
```
a = 'snake'
b = 'language'
max(len(a), len(b))
```
8
8.
```
s = 'python'
s[0:len(s)]
```
'python'
9. `max(1, min(2, 3))` 2

Lists (Solutions)

1. `['eggs', 'bread', 'tea']` `['eggs', 'bread', 'tea']`
2. `nums = [4, 8, 15, 16, 23, 42]`
   `nums[3]` `16`
3. `nums[2:4]` `[15, 16]`
4. `nums[1] = 99`
   `nums` `[4, 99, 15, 16, 23, 42]`
5. `len(nums)` `6`
6. `nest = [[6, 7], [1, 1, 3, 8]]`
   `nest[1][2]` `3`
7. `left = [1, 2, 3]`
   `right = [4, 5, 6]`
   `left + right` `[1, 2, 3, 4, 5, 6]`
8. `left * 2` `[1, 2, 3, 1, 2, 3]`
9. `3 in left` `True`
10. `[2, 3] in left` `False`
11. `alias = left`
    `left[0] = 10`
    `alias` `[10, 2, 3]`